

BASE DE DATOS II — TP2

Food Store

Proyecto integrador — UTN

Integridad, transacciones y concurrencia

Integrante: **Alejo Rayssa** — Equipo G

Fecha: septiembre 2026

Repositorio: <https://github.com/rayssa-ale/food-store>

Parte 0 — Protocolo de seguridad

Protocolo de seguridad — Food Store (TP2, Base de Datos II)

Objetivo: **nunca perder ni corromper los datos del proyecto**. Toda actividad (especialmente cambios estructurales y exports de la herramienta IA) se ejecuta sobre una **copia de trabajo**, dentro de **transacciones**, y con **respaldo previo**.

Entorno real del laboratorio:

- Motor: PostgreSQL 18 local (servicio `postgresql-x64-18`).
- Conexión: `-U postgres -h localhost` (auth `scram-sha-256`).
- `psql` no está en el PATH → ruta completa: `C:\Program Files\PostgreSQL\18\bin\psql.exe` (ídem `pg_dump.exe`, `createdb.exe`).
- Bases: `food_store_prod` (fuente de verdad del negocio) y `food_store_dev` (**copia de trabajo**). Hoy sólo existe la copia.

Paso 1 — Copia: trabajar siempre sobre la copia

Nunca ejecutar DDL ni experimentos sobre la base real. La copia de trabajo se crea clonando el esquema:

```
$psql = 'C:\Program Files\PostgreSQL\18\bin\psql.exe'

# 1a. Clonar la base real (crea food_store_dev idéntica a la fuente)
& $psql -U postgres -h localhost -d postgres -c `
    "CREATE DATABASE food_store_dev TEMPLATE food_store_prod;"

# 1b. (o, si se parte de cero) crear vacía y cargar esquema + datos
& $psql -U postgres -h localhost -d postgres -c "CREATE DATABASE food_store_dev;"
& $psql -U postgres -h localhost -d food_store_dev -f schema.sql
& $psql -U postgres -h localhost -d food_store_dev -f seed.sql
```

Convención usada en el lab: `food_store_dev` = base de trabajo; el `seed.sql` identifica cada prueba con ids explícitos.

Paso 2 — Transacción: probar, medir, y recién entonces COMMIT

Cada cambio se valida dentro de una transacción. Si algo falla o el resultado no es el esperado, `ROLLBACK` y no queda ninguna huella.

```
& $psql -U postgres -h localhost -d food_store_dev -f sql/restricciones.sql
```

El script `sql/restricciones.sql` (Parte 1) se escribió pensando para validarse dentro de la transacción de prueba:

```
BEGIN;
\i sql/restricciones.sql          -- se aplican los triggers
-- ... pruebas con INSERT/UPDATE válidos e inválidos ...
-- si todo fue razonable:
COMMIT;
```

Regla de oro: **una escritura que no se puede revertir no se ejecuta fuera de una transacción supervisada.**

Paso 3 — Respaldo: dump antes de tocar la estructura

Antes de cualquier cambio estructural (y al iniciar la Parte 1) se genera un respaldo de la copia de trabajo:

```
$pgdump = 'C:\Program Files\PostgreSQL\18\bin\pg_dump.exe'
& $pgdump -U postgres -h localhost -d food_store_dev -f `
  "respaldos/food_store_dev_20260922.sql"
```

- Los respaldos viven en `respaldos/` y **no se versionan** en Git.
- Restauración de emergencia: `createdb food_store_dev; psql ... -d food_store_dev -f respaldos/<archivo>.sql`

Cómo se aplicó este protocolo en el TP2

Paso	Evidencia
Copia	DB <code>food_store_dev</code> creada; schema + seed cargados (3/6/3/3/5 filas).
Respaldo	<code>respaldos/food_store_dev_20260922.sql</code> (<code>pg_dump</code> , 10,9 KB) generado antes de la Parte 1.
Transacción	<code>sql/restricciones.sql</code> aplicado y probado dentro de <code>BEGIN/COMMIT</code> ; casos inválidos verificados en transacciones que se revirtieron.
Git	Cada parte con commit descriptivo; la Parte 1 revisó <code>git diff</code> del script antes de commitear.

Parte 1 — Restricciones de integridad (trigger)

```
-- =====
-- FOOD STORE – TP2 (Base de Datos II)
-- Parte 1: Reglas de negocio garantizadas por el motor
-- R8/R9/R10 (ver DUIA parte 1 para la especificación completa)
--
-- Se aplica sobre la COPIA de trabajo:
-- BEGIN; \i sql/restricciones.sql; ... COMMIT;
-- Idempotente: las funciones se reemplazan y los triggers se
-- recrean (DROP IF EXISTS).
-- =====

-- -----
-- R8 – Transiciones de estado del pedido
-- * Un pedido en estado terminal (ENTREGADO, CANCELADO) no
--   puede cambiar de estado.
-- * El estado solo avanza por el flujo
--   PENDIENTE(1) -> EN_PREPARACION(2) -> ENTREGADO(3),
--   o se puede CANCELAR desde cualquier estado no terminal.
--   No se puede volver a un estado anterior.
-- -----

CREATE OR REPLACE FUNCTION check_transicion_estado_pedido()
RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
    v_ordinal_viejo int;
    v_ordinal_nuevo int;
    v_flujo          estado_pedido[] := ARRAY['PENDIENTE', 'EN_PREPARACION', 'ENTREGADO'];
BEGIN
    IF NEW.estado = OLD.estado THEN
        RETURN NEW;
    END IF;

    -- (a) estados terminales congelados
    IF OLD.estado IN ('ENTREGADO', 'CANCELADO') THEN
        RAISE EXCEPTION 'R8: pedido % en estado terminal (%), no puede cambiar a %',
            OLD.id, OLD.estado, NEW.estado
            USING ERRCODE = 'P0001';
    END IF;
```

```

-- (b) cancelación permitida desde cualquier estado no terminal
IF NEW.estado = 'CANCELADO' THEN
    RETURN NEW;
END IF;

-- apilar un estado intermedio se rechaza
SELECT array_position(v_flujo, OLD.estado), array_position(v_flujo, NEW.estado)
    INTO v_ordinal_viejo, v_ordinal_nuevo;

IF v_ordinal_nuevo IS NULL OR v_ordinal_nuevo <= v_ordinal_viejo THEN
    RAISE EXCEPTION 'R8: transición de estado inválida % -> %',
        OLD.estado, NEW.estado
        USING ERRCODE = 'P0001';
END IF;

RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_pedido_estado_transicion ON pedido;
CREATE TRIGGER trg_pedido_estado_transicion
BEFORE UPDATE OF estado ON pedido
FOR EACH ROW EXECUTE FUNCTION check_transicion_estado_pedido();

-- -----
-- R9 – Stock suficiente y descuento de stock al vender
-- Al INSERT de una línea de pedido se verifica que haya stock
-- disponible (producto activo y stock >= cantidad) y se lo
-- descuenta de forma atómica. El UPDATE condicional bloquea la
-- fila del producto, serializando ventas concurrentes del
-- mismo producto (evita sobreventa; ver Parte 2).
-- -----

CREATE OR REPLACE FUNCTION verificar_y_descontar_stock()
RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE producto
        SET stock = stock - NEW.cantidad
        WHERE id = NEW.producto_id
            AND activo = TRUE
            AND stock >= NEW.cantidad;

    IF NOT FOUND THEN

```

```

        RAISE EXCEPTION 'R9: stock insuficiente (o producto inactivo) para producto %, se piden %
unidades',
        NEW.producto_id, NEW.cantidad
        USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$;

```

```

DROP TRIGGER IF EXISTS trg_detalle_stock ON detalle_pedido;
CREATE TRIGGER trg_detalle_stock
BEFORE INSERT ON detalle_pedido
FOR EACH ROW EXECUTE FUNCTION verificar_y_descontar_stock();

```

```

-- -----
-- R10 – Precio unitario de la línea congelado (refuerza R4)
-- El precio_unitario se congela al facturar; una vez creada la
-- línea no se puede modificar. Los descuentos o ajustes se
-- expresan con nuevas líneas, nunca alterando la historia.
-- -----

```

```

CREATE OR REPLACE FUNCTION proteger_precio_unitario()
RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.precio_unitario IS DISTINCT FROM OLD.precio_unitario THEN
        RAISE EXCEPTION 'R10: el precio unitario de la línea % está congelado (R4)',
            OLD.id
            USING ERRCODE = 'P0001';
    END IF;
    RETURN NEW;
END;
$$;

```

```

DROP TRIGGER IF EXISTS trg_detalle_precio_congelado ON detalle_pedido;
CREATE TRIGGER trg_detalle_precio_congelado
BEFORE UPDATE OF precio_unitario ON detalle_pedido
FOR EACH ROW EXECUTE FUNCTION proteger_precio_unitario();

```

DUIA — Parte 1: Restricciones de integridad (TP2 Food Store)

Materia: Base de Datos II — Tecnicatura en Programación (UTN) **Proyecto:** Food Store — TP2, Parte 1 **Tema:** Garantizar reglas de negocio con el motor (triggers) usando OpenCode en modo Plan y revisión humana del diff.

Herramientas usadas

Herramienta	Uso
OpenCode (CLI, modelo *big-pickle)	Motor primario: especificación, diseño de triggers, ejecución controlada y verificación en el motor.
PostgreSQL 18 (local)	Motor donde se aplicaron y verificaron las restricciones sobre la copia <code>food_store_dev</code> .
Git	Versionado por parte; revisión del <code>git diff</code> antes del commit.

Modo de trabajo IA (requisito del TP)

- Modo Plan:** se describió en chat el conjunto de reglas de negocio candidatas y cómo se garantizaría cada una (tabla → trigger).
- Generación:** OpenCode produjo `sql/restricciones.sql` a partir de la especificación aprobada.
- Revisión humana del diff:** se ejecutó `git diff --cached` y se revisó el archivo completo antes de commitear (prueba: `git diff -- cached --stat` más lectura línea por línea).
- Verificación en el motor:** aplicación en transacción + casos válidos e inválidos con salida real de PostgreSQL.

Especificación (prompt / spec aplicados)

# Regla	Regla de negocio	Garantía implementada	Ejemplo de violación que hoy pasaba
R8	El estado del pedido no puede retroceder ni cambiar desde un estado terminal	Trigger <code>trg_pedido_estado_transicion</code> (BEFORE UPDATE OF estado)	<code>UPDATE pedido SET estado='PENDIENTE' WHERE id=2 (EN_PREPARACION)</code>
R9	No se venden más unidades de las	Trigger <code>trg_detalle_stock</code> (BEFORE INSERT) con <code>UPDATE producto ... WHERE stock >= cantidad</code>	<code>INSERT detalle (producto 6, cantidad 9999) con stock 12</code>

# Regla	Regla de negocio	Garantía implementada	Ejemplo de violación que hoy pasaba
	disponibles; el stock se descuenta al vender		
R10	El precio unitario de la línea ya creada es inmodificable (refuerza R4)	Trigger <code>trg_detalle_precio_congelado</code> (BEFORE UPDATE OF <code>precio_unitario</code>)	<code>UPDATE detalle_pedido SET precio_unitario=500 WHERE id=1</code>

Prompt usado (resumen): «Garantizá en el motor las reglas R8 (estados de pedido sin retroceso ni modificación de terminales), R9 (stock suficiente y descuento atómico) y R10 (precio de línea congelado, refuerzo de R4) mediante triggers plpgsql sobre el esquema Food Store. Aplicá dentro de una transacción sobre la copia y probá casos válidos e inválidos.»

Pruebas ejecutadas (salidas reales de PostgreSQL)

Válidas (dentro de la transacción de aplicación, COMMIT final): - v1 PENDIENTE→EN_PREPARACION: `UPDATE 1` . - v2 línea con stock suficiente: `INSERT 0 1` ; stock del producto 5 descontado (30 → **29**) . - v3 pedido PENDIENTE→CANCELADO (no terminal): `INSERT 0 1` + `UPDATE 1` .

Inválidas (cada una en su transacción, ROLLBACK de la tx abortada): - I1 EN_PREPARACION→PENDIENTE → `ERROR R8: transición de estado inválida` . - I2 ENTREGADO→CANCELADO → `ERROR R8: pedido 1 en estado terminal (ENTREGADO)` . - I3 cantidad 9999 con stock 12 → `ERROR R9: stock insuficiente` . - I4 modificar `precio_unitario` → `ERROR R10: ... congelado (R4)` . - Control final: pedidos `4` , detalle total `6` , líneas del pedido `2` intactas (`2`) — no quedaron huellas de los casos rechazados.

Revisión humana aplicada

- Cada trigger se leyó y se justificó contra la regla de negocio antes de ejecutarse (protocolo_seguridad.md).
- El diff del commit (197 líneas sumadas) se revisó antes de versionar.
- Los errores de cada caso inválido fueron **verificados en el motor**, no solo predichos por la IA.

Informe de laboratorio — Concurrencia (TP2, Parte 2)

Materia: Base de Datos II — UTN | **Proyecto:** Food Store **Motor:** PostgreSQL 18 (local) · **Base de trabajo:** `food_store_dev` (protocolo_seguridad.md: toda la actividad sobre la copia)

Contexto y metodología

- **Sesiones:** dos conexiones concurrentes reales al motor (etiquetadas **A** y **B**), sobre las tablas `producto`, `pedido` y `detalle_pedido` con los datos de `seed.sql`.
- **MQVC / aislamiento:** PostgreSQL usa MVCC. El nivel por defecto es `READ COMMITTED` (nueva instantánea por **sentencia**); con `REPEATABLE READ` la instantánea se fija en la **primera sentencia** y se mantiene toda la transacción.
- **Objetivo:** reproducir 3+ anomalías de concurrencia (se hicieron 4), leer la explicación de la IA, y **verificarla contra el motor**, comparando el nivel de aislamiento que la corrige.

Cuadro de la solución esperada (explicación a priori de la IA)

Herramienta usada: OpenCode (CLI de IA), asistente *big-pickle*.

En PostgreSQL, `READ COMMITTED` toma una instantánea nueva por cada sentencia dentro de la transacción. Por eso la misma consulta ejecutada dos veces puede devolver resultados distintos si otra sesión committeó cambios entre medias: eso origina **lecturas no repetibles** (cambio de un valor en particular) y **lecturas fantasma** (cambio en el *set* de filas que matchea un COUNT/SUM). Con `REPEATABLE READ`, en cambio, la transacción queda anclada a una única instantánea tomada en su primera consulta, así que ambas anomalías desaparecen. Por otro lado, una escritura o un `SELECT ... FOR UPDATE` bloquea la fila hasta el COMMIT/ ROLLBACK de su transacción: otra sesión que quiera `UPDATE` de la misma fila **espera**. Si dos sesiones se quedan esperando cada una por una fila que la otra retiene, se produce un **interbloqueo** que el motor detecta y resuelve abortando una de las transacciones (SQLSTATE 40P01).

Escenario	Nivel que reproduce	Nivel que corrige	Mecanismo en PG
1. Lectura no repetible	READ COMMITTED	REPEATABLE READ	instantánea fija
2. Lectura fantasma	READ COMMITTED	REPEATABLE READ	instantánea fija
3. Espera por bloqueo	cualquiera	<code>FOR UPDATE</code> previo + commit temprano	lock de fila
4. Interbloqueo	2 sesiones con locks cruzados	detector de interbloqueo	abort de una tx (40P01)

Escenario 1 — Lectura no repetible

Cómo se reprodujo

Sesión A	Sesión B
BEGIN;	
SELECT precio FROM producto WHERE id=1; → 1200.00	
(idle)	BEGIN;
	UPDATE producto SET precio=1300.00 WHERE id=1;
	COMMIT;
SELECT precio FROM producto WHERE id=1; → ???	

Salida real del motor (transcript)

```
[A] BEGIN (READ COMMITTED, por defecto)
[A] SELECT precio -> 1200.00
[B] UPDATE producto SET precio=1300 WHERE id=1
[B] COMMIT
[A] SELECT precio (de nuevo) -> 1300.00      ← CAMBIÓ
```

Explicación de la IA

La transacción A quedó en `READ COMMITTED`. Su primera lectura tomó una instantánea con 1200.00; la segunda, ejecutada después del `COMMIT` de B, tomó una instantánea nueva y vio 1300.00. Es la definición de lectura no repetible: el mismo `SELECT`, en la misma transacción, con dos resultados. La regla "precio congelado al facturar" (R4/R10) hace relevante esta anomalía: una facturación confiaría en el precio de una lectura anterior y se llevaría una sorpresa.

Verificación en el motor

Se repitió el escenario cambiando el aislamiento de A a `REPEATABLE READ`:

```
[A] BEGIN + SET ISOLATION LEVEL REPEATABLE READ
[A] SELECT precio -> 1300.00
[B] UPDATE producto SET precio=1400 WHERE id=1
[B] COMMIT
[A] SELECT precio (de nuevo) -> 1300.00      ← ESTABLE
```

Confirma la explicación: con `REPEATABLE READ` la segunda lectura sigue viendo la misma instantánea (1300.00) aunque B ya committeó 1400.00.

Escenario 2 — Lectura fantasma

Cómo se reprodujo

Sesión A	Sesión B
<code>BEGIN;</code>	
<code>SELECT count(*) FROM detalle_pedido WHERE pedido_id=1; → 2</code>	
(idle)	<code>BEGIN;</code>
	<code>INSERT INTO detalle_pedido (pedido_id, producto_id, ...) VALUES (1, 5, 1, 500.00);</code>
	<code>COMMIT;</code>
<code>SELECT count(*) FROM detalle_pedido WHERE pedido_id=1; → ???</code>	

Salida real del motor

```
[A] SELECT count(*) lineas pedido 1 -> 2
[B] INSERT detalle (pedido 1, producto 5)
[B] COMMIT
[A] SELECT count(*) (de nuevo) -> 3          ← línea "fantasma"
```

Explicación de la IA

A cuenta las líneas del pedido 1 y obtiene 2. B inserta una línea nueva y committea. La segunda cuenta de A, al tomar una instantánea fresca en `READ COMMITTED`, incluye la fila recién creada: 3. La fila "aparece" en mitad de la transacción de A: es una lectura fantasma. El riesgo concreto: A podría estar recalculando el total del pedido (SUM) para confirmar el cobro y el resultado cambia de una consulta a la otra.

Verificación en el motor

Con A en `REPEATABLE READ`, el mismo COUNT no incorpora la fila de B:

```
[A] SELECT count(*) lineas pedido 1 -> 3
[B] INSERT detalle (pedido 1, producto 3)
```

```
[B] COMMIT
```

```
[A] SELECT count(*) (de nuevo) -> 3
```

```
← ESTABLE
```

Confirma la explicación: por MVCC, PostgreSQL también evita las lecturas fantasma con `REPEATABLE READ` (a diferencia del estándar ANSI, que la prevé solo en `SERIALIZABLE`).

Escenario 3 — Espera por bloqueo (`SELECT ... FOR UPDATE`)

Cómo se reprodujo

Sesión A	Sesión B
<code>BEGIN;</code>	
<code>SELECT stock FROM producto WHERE id=6 FOR UPDATE;</code> → 12	(inicia +2,5 s)
(tiene el lock ~4,5 s)	<code>BEGIN; UPDATE producto SET stock=stock WHERE id=6;</code> → se bloquea
<code>COMMIT;</code> (libera)	→ desbloqueado, continúa
	<code>COMMIT;</code>

Salida real del motor (con tiempos reales)

```
[A] SELECT stock ... id=6 FOR UPDATE (bloqueo adquirido)
[A] sigo con el lock...
[A] COMMIT (libera el lock)
[B] UPDATE producto id=6 -> DESBLOQUEADO, esperó 4.51s
[B] COMMIT
```

Explicación de la IA

`SELECT ... FOR UPDATE` toma un lock de exclusión sobre la fila. B, al intentar actualizar la misma fila, queda encolada detrás de A hasta el `COMMIT` de A (4,51 s en la medición). Es el mecanismo que evita la **sobreventa**: si dos cajeros vendieran el mismo producto, la segunda venta debe esperar a que la primera decida y, al recién liberarse el lock, el stock leído ya refleja la primera venta.

Verificación en el motor

El `UPDATE` de B retornó recién **después** del `COMMIT` de A (4,51 s de espera medida). La fila se desbloquea y la operación completó sin perder datos. Coincide con la explicación. Este mismo `FOR UPDATE` implícito se usa en el trigger R9 (`UPDATE producto ... WHERE stock >= cantidad`) para serializar ventas.

Escenario 4 — Interbloqueo (deadlock, `SQLSTATE 40P01`)

Cómo se reproduce

Ambas sesiones adquieren locks en **orden inverso**:

Sesión A	Sesión B
BEGIN;	(inicia +1 s) BEGIN;
SELECT ... FROM producto WHERE id=1 FOR UPDATE;	SELECT ... FROM producto WHERE id=4 FOR UPDATE;
(espera 3 s) SELECT ... FROM producto WHERE id=4 FOR UPDATE; → espera a B	(espera 3 s) SELECT ... FROM producto WHERE id=1 FOR UPDATE; → espera a A

Ninguna puede avanzar: **interbloqueo**.

Salida real del motor

```
[D:A] BEGIN
[D:A] LOCK producto 1
[D:B] BEGIN
[D:B] LOCK producto 4
[D:A] ABORTADO -> DeadlockDetected sqlstate=40P01
[D:B] LOCK producto 1
[D:B] COMMIT (termine)
```

Explicación de la IA

A retiene el producto 1 y pide el 4; B retiene el producto 4 y pide el 1. Cada una espera el lock que la otra tiene: un ciclo de espera sin salida. PostgreSQL detecta el ciclo con su deadlock detector y aborta una de las transacciones con SQLSTATE **40P01** (DeadlockDetected), dejando que la otra termine. La transacción abortada debe reemitirse (retry) por la sesión que la originó — es la estrategia estándar.

Verificación en el motor

El transcript muestra exactamente lo predicho: la sesión A fue abortada por el motor con `sqlstate=40P01` y la sesión B completó su trabajo y committeó. **Confirmado en el motor.**

Síntesis

Escenario	Anomalía observada	Solución/estado correcto	Evidencia
1. Lectura no repetible	1200.00 → 1300.00	REPEATABLE READ : estable	transcript RC vs RR
2. Lectura fantasma	COUNT 2 → 3	REPEATABLE READ : estable	transcript RC vs RR
3. Espera por bloqueo	B esperó 4,51 s	FOR UPDATE + COMMIT temprano	tiempos reales

Escenario	Anomalía observada	Solución/estado correcto	Evidencia
4. Interbloqueo	ciclo A↔B	detector del motor: 40P01	sqlstate capturado

Decisiones tomadas:

- 1. En los flujos de facturación (leer precio + sumar líneas) usar `REPEATABLE READ` para que el "precio congelado" (R4/R10) sea perceptivamente correcto al momento de cobrar.
- 2. Para inventario (R9) mantener el `UPDATE ... WHERE stock >= cantidad` que ya serializa por fila; nada de `SELECT` puro + `UPDATE` por arriba.
- 3. Ante 40P01, releer el estado y reemitir la transacción abortada.
- 4. Todo se trabajó sobre la copia `food_store_dev`, en sesiones de prueba; los datos quedaron consistentes al final del lab (precio=1200.00, stock p6=12, 2 líneas en pedido 1).

DUIA — Parte 2: Laboratorio de concurrencia (TP2 Food Store)

Materia: Base de Datos II — UTN · **Proyecto:** Food Store **Tema:** Reproducir anomalías de concurrencia con dos sesiones, explicación generada por IA y verificación en el motor.

Herramientas usadas

Herramienta	Uso
OpenCode (CLI de IA, modelo *big-pickle)	Predicción/explicación de cada anomalía y de la solución (por escenario).
PostgreSQL 18 (local)	Motor donde se reprodujeron y verificaron los 4 escenarios (<code>food_store_dev</code>).
Python 3.13 + psycopg2	Control fino de dos sesiones concurrentes (inicio, pausas, captura de tiempos y SQLSTATE).
Git	Versionado del informe y la DUIA.

Para qué se usó la IA y con qué prompts

Uso	Prompt / especificación aplicados
Predecir qué anomalía produce cada escenario y qué nivel de aislamiento la corrige	«En PostgreSQL, describí qué pasa con SELECT repetidos entre dos sesiones en READ COMMITTED vs REPEATABLE READ para precio (no repetible) y COUNT (fantasma); y qué pasa con FOR UPDATE cruzado (espera y 40P01). Explicá por qué el motor prefiere abortar una tx»
Redactar explicaciones por escenario	Se copiaron textuales en <code>docs/informe_concurrencia.md</code> y luego se confrontaron con la salida real del motor .

Verificación (regla del TP: la IA propone, el motor decide)

Cada explicación se confrontó con salidas reales:

- No repetible:** en READ COMMITTED el precio cambió 1200→1300; en REPEATABLE READ quedó estable (1300→1300). ✓
- Fantasma:** COUNT 2→3 en RC; estable (3→3) en RR. ✓
- Espera:** sesión B esperó **4,51 s** medidos hasta el COMMIT de A. ✓
- Interbloqueo:** `DeadlockDetected sqlstate=40P01` capturado; la otra sesión committeó. ✓

Revisión humana aplicada

- Los 4 escenarios se ejecutaron sobre la copia `food_store_dev` (protocolo_seguridad.md), sin tocar datos productivos.
- Se validó la consistencia final de la base (precio p1 1200.00, stock p6 12, 2 líneas en pedido 1).
- Los tiempos y SQLSTATE del informe provienen del transcript del motor, no de la IA.

Parte 3 — Ejercicio de lectura crítica

Ejercicio de lectura crítica (TP2, Parte 3)

Materia: Base de Datos II — UTN · **Proyecto:** Food Store **Ejercicio:** dos scripts SQL aparentemente simples pero peligrosos. Para cada uno: **qué hace**, **por qué es peligroso**, **cómo se corrige**. Cierre: patrón del error y cómo se relaciona con el protocolo de seguridad y las reglas R4–R7.

Script 1 — Baja masiva sin condición

1a. Qué hace

```
UPDATE funcion SET activa = FALSE;
```

Intento (según la consigna): *dar de baja las funciones de las películas que se retiraron de cartel*. El entrenador lo escribió creyendo que ese era el contexto implícito del UPDATE.

1b. Por qué es peligroso

1. **No tiene WHERE** : actualiza TODAS las filas de `funcion`, no solo las de películas retiradas. Si la tabla tiene 40.000 funciones, las 40.000 quedan con `activa = FALSE`.
2. **Destruye información histórica/presente**: mata también funciones futuras ya programadas y en cartel → el cine pierde la grilla; y funciones pasadas cuyas entradas ya se vendieron quedan marcadas "inactivas", cortando la trazabilidad de ventas (equivalente a romper R7 en nuestro esquema: nunca se pierde la historia).
3. **Sin protocolo**: si se ejecuta fuera de una transacción, cada fila se commitea sola y es **irreversible**. No hay `BEGIN/ROLLBACK` para revisar cuántas filas iba a tocar.

Esto es una variante del clásico *UPDATE sin WHERE*: el riesgo no está en la sintaxis (válida) sino en la intención asumida que el motor no puede adivinar.

1c. Cómo se corrige

Hay que **explicitar qué es "retirada de cartel"** y filtrar por eso:

```
BEGIN;

-- control ANTES de escribir: ¿cuáles y cuántas serían afectadas?
SELECT count(*) AS funciones_inactivas_objetivo
FROM funcion f
JOIN pelicula p ON p.id = f.pelicula_id
```

```

WHERE p.retirada_de_cartel = TRUE
      AND f.activa = TRUE;

-- el UPDATE, ahora SI filtrado por la condición
UPDATE funcion f
      SET activa = FALSE
WHERE f.pelicula_id IN (SELECT id FROM pelicula WHERE retirada_de_cartel = TRUE);

-- reviso el resultado antes de hacerlo definitivo
SELECT count(*) AS funciones_ahora_inactivas FROM funcion WHERE activa = FALSE;

COMMIT;  -- solo si los números son razonables; si no: ROLLBACK

```

Regla extra aplicable al proyecto Food Store: como la baja es lógica, mantener un NULL/condición por fila y nunca borrar — y medir el alcance con un `SELECT` del mismo `WHERE` **antes** del `UPDATE`.

Script 2 — `DELETE` con la trampa del `NULL` en `NOT IN`

2a. Qué hace

```
DELETE FROM categoria WHERE id NOT IN (SELECT categoria_id FROM producto);
```

Intento: *borrar las categorías sin productos* (limpieza de datos huérfanos).

2b. Por qué es peligroso

1. **La trampa del `NULL` en `NOT IN`**. Si `producto.categoria_id` puede ser `NULL` en el modelo (categoría opcional), la subconsulta devuelve un conjunto que **contiene `NULL`**. En lógica de tres valores, si `id NOT IN (... , NULL, ...)` → `UNKNOWN` → la condición es falsa para **todas** las categorías → el `DELETE` borra **cero filas**, sin error y sin aviso. El "borrado de categorías vacías" simplemente no ocurre y nadie lo nota.
2. **Borrado físico sobre una tabla maestra**: aunque no hubiera `NULL` y funcionara, elimina físicamente filas de `categoria`, una tabla con historial. En el proyecto Food Store esto viola R7 (sin borrados físicos de categorías). El dato muere para siempre.
3. **Carrera (conurrencia)**: entre la evaluación del `NOT IN` y el `DELETE` otra sesión puede insertar un producto en esa categoría aparentemente vacía → se destruye una categoría recién usada.

2c. Cómo se corrige

`NOT EXISTS` es inmune al `NULL` y refuerza la intención:

```
BEGIN;
```

```

-- control: categorías que quedarían marcadas (sin productos)
SELECT c.id, c.nombre
FROM categoria c
WHERE NOT EXISTS (SELECT 1 FROM producto p WHERE p.categoria_id = c.id);

-- baja lógica (recomendado, respeta R7)
UPDATE categoria c
    SET activo = FALSE
WHERE NOT EXISTS (SELECT 1 FROM producto p WHERE p.categoria_id = c.id);

-- si el negocio exigiera el borrado físico, al menos con NOT EXISTS
-- y dentro de la transacción:
-- DELETE FROM categoria c
-- WHERE NOT EXISTS (SELECT 1 FROM producto p WHERE p.categoria_id = c.id);

-- verificación del alcance antes de COMMIT
SELECT count(*) AS categorias_inactivas FROM categoria WHERE activo = FALSE;

COMMIT;  -- o ROLLBACK si el número no es el esperado

```

Notas de la corrección:

- `NOT EXISTS (SELECT 1 FROM producto p WHERE p.categoria_id = c.id)` **correlaciona** por la fila `c` y no tiene el problema de NULL.
- La transacción **y el re-chequeo en el mismo WHERE** cubren la carrera de la otra sesión (aunque para blindarla del todo se usaría `SELECT ... FOR UPDATE` de las filas de `categoria`, tema de la Parte 2).
- Bajar lógicamente con `activo = FALSE` (esquema Food Store) preserva la historia en lugar de destruirla.

Cierre: patrón de error y vínculo con el protocolo

Script	Patrón	Consecuencia si se ejecuta tal cual	Corrección
1	<code>UPDATE</code> sin <code>WHERE</code> (intención no expresada)	Toda la tabla inactivada; historia perdida; sin vuelta atrás	filtrar por la condición, control <code>SELECT</code> previo, <code>BEGIN/COMMIT</code>
2	<code>NOT IN</code> + subconsulta con NULL	Script silencioso: no borra nada; o borrado físico de maestras	<code>NOT EXISTS</code> , baja lógica, transacción con verificación

Ambos se escribirían igual por un humano apurado que por una IA (predicción plausible del "siguiente token" sin contexto); por eso el mismo protocolo de la Parte 1 aplica aquí:

1. **Copia** – ejecutar siempre sobre `food_store_dev`.
2. **Transacción** – `BEGIN`, medir alcance (`SELECT` con el mismo `WHERE`), decidir, `COMMIT` / `ROLLBACK`.
3. **Respaldo** – `pg_dump` previo a cualquier operación masiva (`respaldos/`).

El motor nunca puede adivinar la intención: un `UPDATE / DELETE` sin condición se ejecuta. La responsabilidad es de quien lo escribe, y la evidencia de la Parte 1 (triggers + transacciones) son las defensas del proyecto contra estos fallos.

DUIA — Parte 3: Ejercicio de lectura crítica (TP2 Food Store)

Materia: Base de Datos II — UTN · **Proyecto:** Food Store **Tema:** Análisis de dos scripts SQL peligrosos (baja masiva sin WHERE y DELETE con trampa de NULL en NOT IN) y sus correcciones.

Herramientas usadas

Herramienta	Uso
OpenCode (CLI de IA, modelo *big-pickle)	Asistencia para identificar los patrones de error (UPDATE sin filtro; lógica de tres valores del NULL en NOT IN) y proponer correcciones.
PostgreSQL 18 (local)	Verificación de la lógica de las correcciones (NOT EXISTS vs NOT IN con NULL) sobre la copia food_store_dev , en caso de duda, dentro de transacciones reversibles.
Git	Versionado del ejercicio y la DUIA.

Para qué se usó la IA y con qué prompts

Script	Uso de la IA	Prompt / especificación
<code>UPDATE funcion SET activa = FALSE</code>	Detectar que la intención (bajas de películas retiradas) no está expresada en la SQL y que falta todo filtro	«Este script debería dar de baja solo funciones de películas retiradas de cartel. Analízalo paso a paso: qué hace, por qué es peligroso y qué corrección hay que aplicar, con control antes de commitear»
<code>DELETE FROM categoria WHERE id NOT IN (SELECT categoria_id FROM producto)</code>	Explicar la trampa del NULL en NOT IN (lógica de tres valores) y la alternativa NOT EXISTS	«Explicá por qué este DELETE puede no borrar nada si categoria_id admite NULL, y mostrá la corrección con NOT EXISTS respetando la baja lógica (R7)»

Revisión humana aplicada

- La corrección NO se limitó a lo que la IA dictó: se cotejó contra el esquema real del Food Store. La que involucraba un DELETE físico sobre `categoria` se descartó en favor de baja lógica `activo = FALSE` (regla R7 del proyecto), decisión humana documentada en el ejercicio.
- Las dos soluciones quedan dentro del protocolo (protocolo_seguridad.md): copia + transacción + control de alcance antes del COMMIT.

- El documento incluye la lógica de tres valores del NULL, verificable en cualquier motor SQL, como conocimiento de la cátedra y no como afirmación aislada de la IA.

Anexo A — Esquema de la base (schema.sql)

```
-- =====
-- FOOD STORE – Proyecto integrador (Base de Datos I, UTN)
-- TP1 – Parte 4: DDL definitivo en PostgreSQL
-- Esquema conciliado entre el modelo relacional (Parte 2) y el
-- resultado de la normalización (Parte 3).
-- PostgreSQL 18+. Ejecutar completo, por ejemplo:
-- \i 'C:/.../schema.sql'
-- =====

-- -----
-- Dominios cerrados (tipos enumerados)
-- -----

-- Formas de pago que usa el negocio (R3 / datos de la planilla).
CREATE TYPE forma_pago AS ENUM ('EFFECTIVO', 'TARJETA', 'TRANSFERENCIA');

-- Ciclo de vida de un pedido (atributo adicional razonable,
-- documentado en el diccionario de datos): permite seguir el
-- pedido sin recurrir a borrados; la baja se expresa con el
-- estado CANCELADO.
CREATE TYPE estado_pedido AS ENUM ('PENDIENTE', 'EN_PREPARACION', 'ENTREGADO', 'CANCELADO');

-- -----
-- cliente
-- -----
CREATE TABLE cliente (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre      VARCHAR(80) NOT NULL,
    apellido    VARCHAR(80) NOT NULL,
    email       VARCHAR(120) NOT NULL,
    telefono    VARCHAR(30),
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    -- R6: el mail identifica de forma única al cliente en el
    -- sistema (clave candidata alternativa de la Parte 1).
    CONSTRAINT uq_cliente_email UNIQUE (email)
);

-- -----
-- categoria
-- -----
```

```

CREATE TABLE categoria (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre      VARCHAR(80) NOT NULL,
    descripcion VARCHAR(500),
    activo      BOOLEAN      NOT NULL DEFAULT TRUE,
    created_at  TIMESTAMPTZ  NOT NULL DEFAULT now(),
    -- Nombres de categoría sin duplicados: evita dos categorías
    -- homónimas ("Bebidas", "Bebidas"). Decisión de diseño a
    -- partir de la presentación del negocio.
    CONSTRAINT uq_categoria_nombre UNIQUE (nombre)
);

```

```

-- -----
-- producto
-- -----

```

```

CREATE TABLE producto (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre      VARCHAR(120) NOT NULL,
    descripcion VARCHAR(500),
    precio      NUMERIC(12,2) NOT NULL,
    stock       INTEGER      NOT NULL DEFAULT 0,
    categoria_id BIGINT      NOT NULL,
    activo      BOOLEAN      NOT NULL DEFAULT TRUE,
    created_at  TIMESTAMPTZ  NOT NULL DEFAULT now(),
    -- R1: todo producto pertenece exactamente a una categoría
    -- (participación total).
    -- ON DELETE RESTRICT: R7 prohíbe eliminar físicamente
    -- categorías; se da de baja con activo = FALSE. Un SELECT
    -- que borre una categoría con productos se rechaza.
    CONSTRAINT fk_producto_categoria FOREIGN KEY (categoria_id)
        REFERENCES categoria (id) ON DELETE RESTRICT,
    -- R5: el precio y el stock nunca son negativos.
    CONSTRAINT ck_producto_precio_no_negativo CHECK (precio >= 0),
    CONSTRAINT ck_producto_stock_no_negativo  CHECK (stock >= 0)
);

```

```

-- -----
-- pedido
-- -----

```

```

CREATE TABLE pedido (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    fecha      TIMESTAMPTZ  NOT NULL DEFAULT now(),
    cliente_id BIGINT      NOT NULL,

```

```

forma_pago forma_pago NOT NULL,
estado estado_pedido NOT NULL DEFAULT 'PENDIENTE',
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
-- R2: todo pedido pertenece exactamente a un cliente registrado
-- (participación total).
-- ON DELETE RESTRICT: un cliente con pedidos no se elimina
-- físicamente; se conserva el historial de ventas completo.
CONSTRAINT fk_pedido_cliente FOREIGN KEY (cliente_id)
REFERENCES cliente (id) ON DELETE RESTRICT
);
-- NOTA de diseño: no se almacena "total" del pedido. Es 100 %
-- derivable de sus líneas (SUM(cantidad * precio_unitario)).
-- Almacenarlo introduciría una dependencia transitiva (viola 3FN,
-- ver Parte 3) y no aporta trazabilidad: el precio histórico que
-- importa (precio_unitario por línea) ya se conserva.

-- -----
-- detalle_pedido (entidad asociativa de la relación N:M)
-- -----

CREATE TABLE detalle_pedido (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    pedido_id BIGINT NOT NULL,
    producto_id BIGINT NOT NULL,
    cantidad INTEGER NOT NULL,
    precio_unitario NUMERIC(12,2) NOT NULL,
    -- Clave candidata natural: el par (pedido_id, producto_id) no
    -- se repite (dentro de un mismo pedido, un producto no se pide
    -- en más de una línea – regla de la planilla, Parte 3). Se
    -- declara UNIQUE y se elige una clave sustituta (id) como PK
    -- para que futuras tablas (descuentos, observaciones por
    -- línea) puedan referenciar la línea con una sola FK.
    CONSTRAINT uq_detalle_pedido_producto UNIQUE (pedido_id, producto_id),
    -- R4: el precio unitario se congela al momento de facturar; por
    -- eso vive en la línea y no en producto.precio (que puede
    -- cambiar). Los montos usan NUMERIC (precisión fija), nunca
    -- punto flotante.
    -- ON DELETE RESTRICT: no se borra un pedido con líneas, ni un
    -- producto que ya se vendió: la historia no se pierde (R7).
    CONSTRAINT fk_detalle_pedido FOREIGN KEY (pedido_id)
    REFERENCES pedido (id) ON DELETE RESTRICT,
    CONSTRAINT fk_detalle_producto FOREIGN KEY (producto_id)
    REFERENCES producto (id) ON DELETE RESTRICT,
    -- R4/R5: las cantidades pedidas son positivas y el precio

```

```

-- unitario de la línea no es negativo.
CONSTRAINT ck_detalle_cantidad_positiva      CHECK (cantidad > 0),
CONSTRAINT ck_detalle_precio_unitario_no_neg CHECK (precio_unitario >= 0)
);

-- -----
-- Índices (justificados por el uso esperado de las tablas)
-- -----

-- 1. Acelera "listar / sumar los pedidos de un cliente"
--   (SELECT ... WHERE cliente_id = ?) y el agrupamiento por
--   cliente en reportes de facturación.
CREATE INDEX idx_pedido_cliente ON pedido (cliente_id);

-- 2. Acelera "listar productos vigentes de una categoría"
--   (SELECT ... WHERE categoria_id = ? AND activo = TRUE).
--   Índice parcial: solo indexa productos activos, que son los
--   que muestran la carta; no se gasta espacio con inactivos.
CREATE INDEX idx_producto_categoria ON producto (categoria_id)
WHERE activo = TRUE;

-- 3. Acelera "cuánto se vendió de cada producto" (SUM por
--   producto) y permite reconstruir la N:M desde el lado producto
--   (detalles de un producto para hallar sus pedidos).
CREATE INDEX idx_detalle_producto ON detalle_pedido (producto_id);

```

Anexo B — Datos de carga inicial (seed.sql)

```
-- =====
-- FOOD STORE – Datos de carga inicial (seed)
-- Base de Datos I / II (UTN)
-- Carga de ejemplo mínima para los TP2 de concurrencia e
-- integridad: categorías, productos, clientes, pedidos y líneas.
-- Debe ejecutarse DESPUÉS de schema.sql, sobre la COPIA de
-- trabajo (ver protocolo_seguridad.md). Ids explícitos con
-- OVERRIDING SYSTEM VALUE para poder referenciarlos con
-- precisión en los labs.
-- =====

BEGIN;

INSERT INTO categoria (id, nombre, descripcion, activo) OVERRIDING SYSTEM VALUE VALUES
    (1, 'Pizzas', 'Pizzas artesanales a la piedra', TRUE),
    (2, 'Bebidas', 'Gaseosas, aguas y cervezas', TRUE),
    (3, 'Postres', 'Postres caseros del día', TRUE);

INSERT INTO producto (id, nombre, descripcion, precio, stock, categoria_id, activo) OVERRIDING SYSTEM
VALUE VALUES
    (1, 'Muzzarella', 'Grande, salsa y queso muzzarella', 1200.00, 10, 1, TRUE),
    (2, 'Napolitana', 'Grande, tomate y ajo', 1500.00, 8, 1, TRUE),
    (3, 'Jamón y Queso', 'Grande, jamón cocido y queso', 1400.00, 5, 1, TRUE),
    (4, 'Coca-Cola 1L', 'Gaseosa cola, 1 litro', 850.00, 50, 2, TRUE),
    (5, 'Agua mineral 500ml', 'Agua sin gas, botella PET', 500.00, 30, 2, TRUE),
    (6, 'Flan con dulce de leche', 'Flan casero, porción', 950.00, 12, 3, TRUE);

INSERT INTO cliente (id, nombre, apellido, email, telefono) OVERRIDING SYSTEM VALUE VALUES
    (1, 'Dana', 'Da Luz', 'dana.daluz@gmail.com', '3511234567'),
    (2, 'Carlos', 'Abregú', 'carlos.abregu@outlook.com', '3519876543'),
    (3, 'Ramiro', 'Fariña', 'ramiro.farina@gmail.com', '3515551122');

INSERT INTO pedido (id, fecha, cliente_id, forma_pago, estado) OVERRIDING SYSTEM VALUE VALUES
    (1, '2026-09-20 20:15:00-03', 1, 'TARJETA', 'ENTREGADO'),
    (2, '2026-09-21 21:30:00-03', 2, 'EFECTIVO', 'EN_PREPARACION'),
    (3, now(), 3, 'TRANSFERENCIA', 'PENDIENTE');

INSERT INTO detalle_pedido (id, pedido_id, producto_id, cantidad, precio_unitario) OVERRIDING SYSTEM
VALUE VALUES
    (1, 1, 1, 2, 1200.00),
```

```
(2, 1, 4, 1, 850.00),  
(3, 2, 2, 1, 1500.00),  
(4, 2, 5, 2, 500.00),  
(5, 3, 6, 3, 950.00);
```

```
-- Sincroniza las secuencias identity después de insertar ids fijos,  
-- para que el próximo GENERATED ALWAYS AS IDENTITY no colisione.
```

```
SELECT setval(pg_get_serial_sequence('categoria', 'id'), (SELECT max(id) FROM categoria));  
SELECT setval(pg_get_serial_sequence('producto', 'id'), (SELECT max(id) FROM producto));  
SELECT setval(pg_get_serial_sequence('cliente', 'id'), (SELECT max(id) FROM cliente));  
SELECT setval(pg_get_serial_sequence('pedido', 'id'), (SELECT max(id) FROM pedido));  
SELECT setval(pg_get_serial_sequence('detalle_pedido', 'id'), (SELECT max(id) FROM detalle_pedido));
```

```
COMMIT;
```

Anexo C — Pruebas válidas de restricciones

```
-- =====
-- FOOD STORE – Pruebas de restricciones (Parte 1)
-- PROTOCOLO: se aplican los triggers y se prueban los casos
-- VÁLIDOS dentro de UNA transacción, y recién al final COMMIT.
--
-- Ejecución:
--   psql -d food_store_dev -f sql/pruebas_validas_restricciones.sql
-- =====

BEGIN;

-- Aplica las restricciones R8, R9 y R10 (script entregado).
\i sql/restricciones.sql

-- -----
-- Casos VÁLIDOS (deben ejecutarse sin error)
-- -----

-- V1: avance de estado permitido PENDIENTE -> EN_PREPARACION (pedido 3).
UPDATE pedido SET estado = 'EN_PREPARACION' WHERE id = 3;

-- V2: carga de línea con stock suficiente: producto 5 (stock 30),
--     se piden 1 unidad. La venta debe descontar el stock a 29.
INSERT INTO detalle_pedido (pedido_id, producto_id, cantidad, precio_unitario)
VALUES (3, 5, 1, 500.00);

-- V3: pedido nuevo en PENDIENTE se puede CANCELAR (estado no terminal).
INSERT INTO pedido (id, fecha, cliente_id, forma_pago, estado)
OVERRIDING SYSTEM VALUE
VALUES (4, now(), 3, 'TRANSFERENCIA', 'PENDIENTE');
UPDATE pedido SET estado = 'CANCELADO' WHERE id = 4;

-- Verificación de stock descontado: el producto 5 debe quedar en 29.
SELECT id, nombre, stock FROM producto WHERE id = 5;

COMMIT;
```

Anexo D — Casos inválidos de restricciones

```
-- =====
-- FOOD STORE – Casos INVÁLIDOS (Parte 1)
-- Cada caso corre dentro de SU PROPIA transacción. El error que
-- dispara el trigger deja la transacción abortada y se hace
-- ROLLBACK. La existencia del ERROR es la evidencia de que la
-- regla quedó garantizada por el motor.
-- =====

-- I1 – R8: no se puede volver de EN_PREPARACION a PENDIENTE.
BEGIN;
UPDATE pedido SET estado = 'PENDIENTE' WHERE id = 2;
ROLLBACK;

-- I2 – R8: un pedido ENTREGADO está congelado (no se puede cancelar).
BEGIN;
UPDATE pedido SET estado = 'CANCELADO' WHERE id = 1;
ROLLBACK;

-- I3 – R9: no se venden más unidades de las disponibles
--      (producto 6 tiene stock 12; se piden 9999).
BEGIN;
INSERT INTO detalle_pedido (pedido_id, producto_id, cantidad, precio_unitario)
VALUES (2, 6, 9999, 950.00);
ROLLBACK;

-- I4 – R10: el precio unitario de una línea ya creada no se modifica.
BEGIN;
UPDATE detalle_pedido SET precio_unitario = 500.00 WHERE id = 1;
ROLLBACK;

-- Control final: los datos quedan intactos luego de los rechazos.
SELECT (SELECT count(*) FROM pedido)           AS pedidos,
       (SELECT count(*) FROM detalle_pedido)    AS detalle_total,
       (SELECT count(*) FROM detalle_pedido WHERE pedido_id = 2) AS lineas_pedido_2;
```